



Faculty of Computers and Data Science
Intelligent Systems Department
NLP Project
Mail spam classification



Team Members

<i>Name</i>	<i>ID</i>
<i>Ahmed Mostafa AbdelRahman</i>	<i>20221372883</i>
<i>Mazen Gaber Mahmoud</i>	<i>20221372110</i>

Contents

1 Introduction	4
1.1 Project Idea	4
1.2 Data Used	4
1.2.1 Source	4
1.2.2 Data Description	4
1.3 Objectives	5
2 Text preprocessing	5
2.1 Removing HTML Tags	5
2.2 Tokenization	5
2.3 Removing Stop Words	5
2.4 Stemming	5
3 Data preprocessing	6
3.1 Data Splitting	6
3.2 Text Vectorization	6
4 Model Training	7
4.1 Naive Bayes and Logistic Regression Classification	7
4.2 RNN Models (Simple RNN and LSTM)	7
5 Results	10
5.1 Naïve Bayes	10
5.2 Logistic Regression	12
5.3 Simple RNN	13
5.4 LSTM	14
6 Test	15

1 Introduction

Mail spam classification is really important for keeping our emails safe and organized. It helps to find and stop bad or unwanted emails, like ones that try to trick us into giving personal information or clicking on harmful links. By doing this, it makes sure we only get the emails we really want and keeps our inbox clean and safe. This helps us stay protected while using email and makes it easier for us to see and respond to the emails that matter to us.

1.1 Project Idea

Comparing the performance of different models in classifying whether mail is spam or not

1. Naive Bayes
2. Logistic Regression
3. LSTM
4. Simple RNN

1.2 Data Used

Spam Classification for Basic NLP

1.2.1 Source

Kaggle

1.2.2 Data Description

This Data consist of raw mail messages which is suitable for the NLP pre-processing like Tokenizing, Removing Stop words, Stemming and Parsing HTML tags.

1.3 Objectives

- Explore traditional machine learning approaches for spam classification.
- Evaluate recurrent neural network models for more complex feature extraction.
- Compare the performance of different models and identify the most effective model.

2 Text preprocessing

2.1 Removing HTML Tags

This step involves eliminating HTML tags present in the text data, by using regular expressions to ensure the content is in a readable text format without any HTML markup.

2.2 Tokenization

Tokenization breaks down the text into smaller units, such as words or phrases (tokens). It splits the text into individual meaningful units to facilitate further processing.

2.3 Removing Stop Words

Stop words are common words (like "the", "is", "at", etc.) that often don't contribute significantly to the meaning of the text. Removing these words can help in reducing noise and improving the efficiency of analysis or model training.

2.4 Stemming

Stemming involves reducing words to their root or base form. For instance, words like "running," "runs," and "ran" might all be reduced

to the stem "run." This process helps in making similar words to a common form, which can assist in text analysis or classification tasks.

3 Data preprocessing

3.1 Data Splitting

The dataset is divided into three separate subsets:

- 1. Training Set:**

Contains a significant portion (80%) of the original dataset.

- 2. Validation Set:**

Represents a smaller portion (10%) of the original dataset.

- 3. Testing Set:**

Also contains a smaller portion (10%) of the original dataset.

3.2 Text Vectorization

To perform tasks on textual data, it needs to be converted into a numerical format. This process, known as text vectorization, is accomplished using CountVectorizer:

- CountVectorizer:**

- Converts text data into a numerical matrix format.
- The training set's textual content is transformed into a matrix of token counts.
- The vocabulary learned from the training data is used to represent the validation and testing data in the same numerical format. This ensures consistency in the way textual data is represented across all sets while preserving the relationship between words and their counts.

4 Model Training

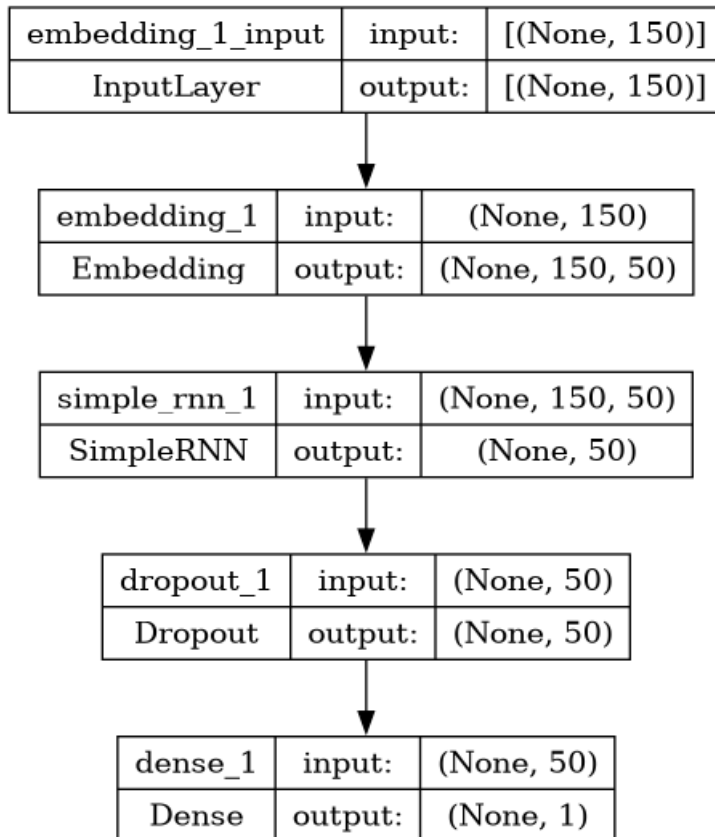
4.1 Naive Bayes and Logistic Regression Classification

- Naive Bayes Classifier:
 1. Trains a Naive Bayes classifier using the Multinomial Naive Bayes algorithm.
 2. Uses the count of words to make predictions based on probability. Trained using the training set, then predicts labels for the test set.
- Logistic Regression:
 1. Trains a Logistic Regression classifier.
 2. Makes predictions on the test set and calculates accuracy.

4.2 RNN Models (Simple RNN and LSTM)

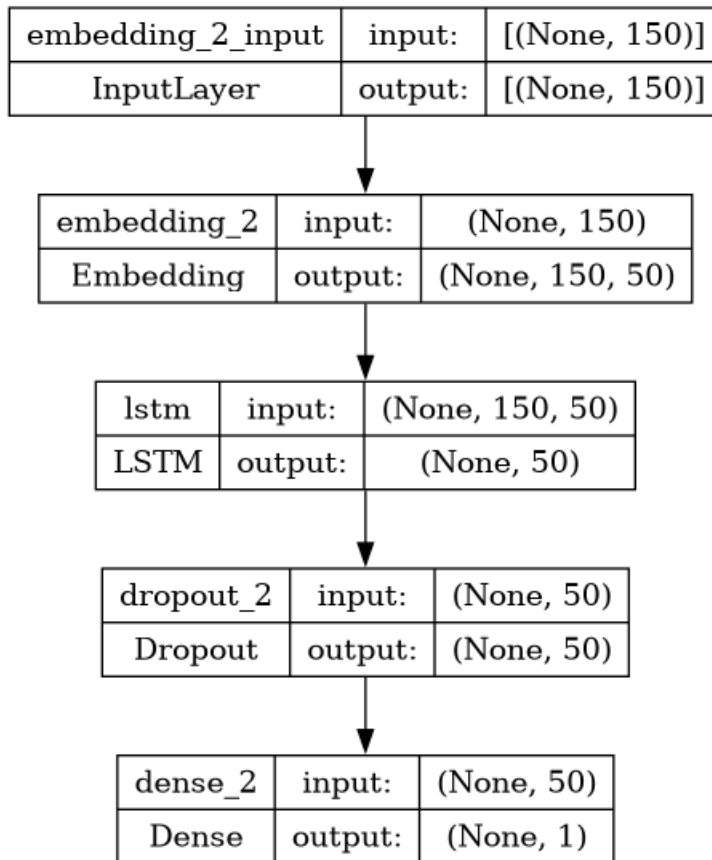
- Tokenizes text data and converts it into numerical sequences for RNN input.
- Pads sequences to ensure a fixed length for RNN models.

Simple RNN



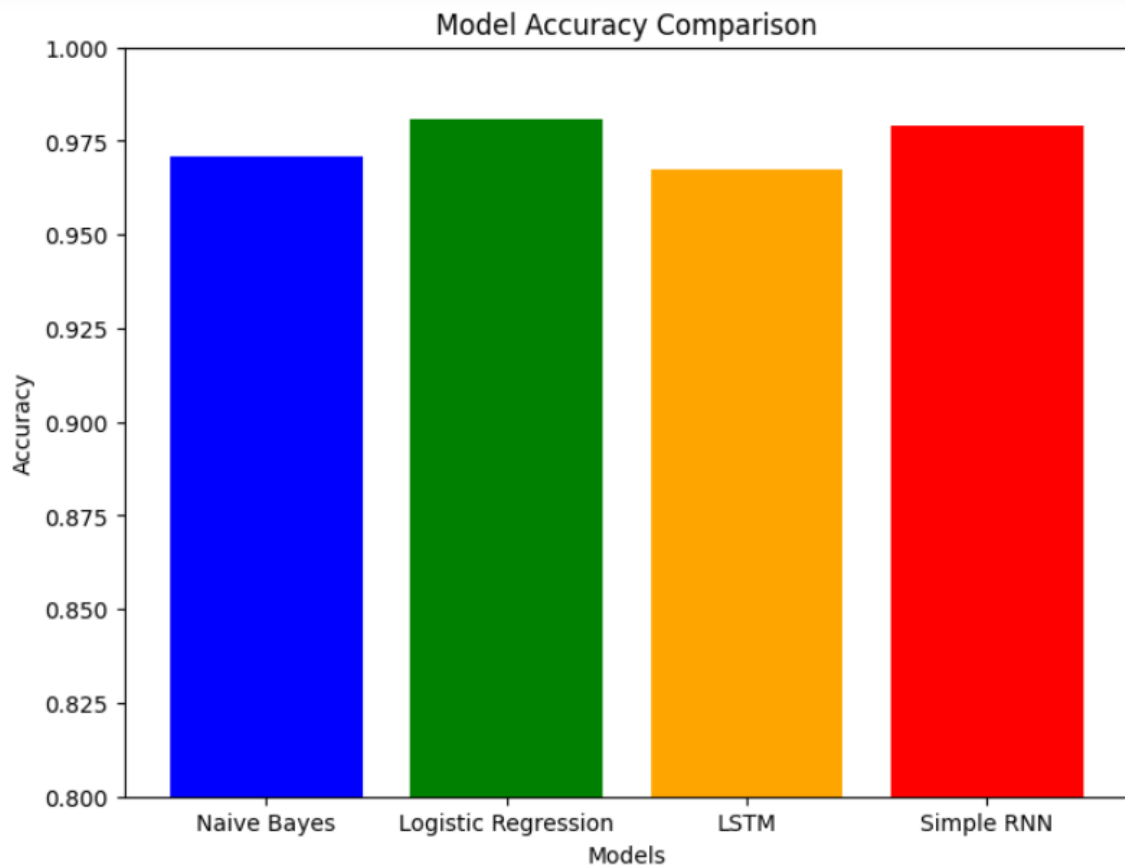
- **Embedding Layer:** Represents the initial layer converting words into fixed-size dense vectors. Each word is mapped to a high-dimensional vector.
- **Simple RNN Layer:** Shows the Simple RNN layer with 50 units, describing connections between input and output.
- **Dropout Layer:** Illustrates the dropout layer, indicating the dropout rate of 0.5 (50% dropout).
- **Dense Layer:** Represents the output layer with a single neuron using a sigmoid activation function for binary classification.

LSTM model



- **Embedding Layer:** Represents the initial layer converting words into fixed-size dense vectors. Each word is mapped to a high-dimensional vector.
- **LSTM Layer:** Shows the LSTM layer with 50 units, describing connections between input, output, and memory cell states.
- **Dropout Layer:** Illustrates the dropout layer, indicating the dropout rate of 0.5 (50% dropout).
- **Dense Layer:** Represents the output layer with a single neuron using a sigmoid activation function for binary classification.

5 Results



5.1 Naïve Bayes

Naive Bayes Accuracy: 0.9706896551724138

Naive Bayes Classification Report:

	precision	recall	f1-score	support
0	0.96	0.99	0.98	376
1	0.99	0.93	0.96	204
accuracy			0.97	580
macro avg	0.98	0.96	0.97	580
weighted avg	0.97	0.97	0.97	580

Naive Bayes Accuracy:

- **Accuracy:** 97.07%
 - This percentage indicates the overall correctness of the predictions made by the Naive Bayes classifier on the test dataset.

Classification Report:

- **Precision:** Indicates the ratio of correctly predicted instances of a class to the total instances predicted as that class.
 - For class "0" (Not spam): Precision is 96%, meaning 96% of the predicted "Not spam" instances were actually "Not spam."
 - For class "1" (Spam): Precision is 99%, indicating 99% of the predicted "Spam" instances were actually "Spam."
- **Recall (Sensitivity):** Denotes the ratio of correctly predicted instances of a class to the actual instances of that class in the dataset.
 - For class "0" (Not spam): Recall is 99%, signifying that 99% of the actual "Not spam" instances were correctly identified.
 - For class "1" (Spam): Recall is 93%, indicating that 93% of the actual "Spam" instances were correctly identified.
- **F1-score:** The mean of precision and recall, providing a balanced assessment of the classifier's performance for each class.
 - For class "0" (Not spam): F1-score is 98%.
 - For class "1" (Spam): F1-score is 96%.

- **Support:** Indicates the number of occurrences of each class in the test set.
 - For class "0" (Not spam): There are 376 instances.
 - For class "1" (Spam): There are 204 instances.

5.2 Logistic Regression

Logistic Regression Accuracy: 0.9810344827586207

Logistic Regression Classification Report:

	precision	recall	f1-score	support
0	0.99	0.98	0.99	376
1	0.97	0.98	0.97	204
accuracy			0.98	580
macro avg	0.98	0.98	0.98	580
weighted avg	0.98	0.98	0.98	580

- The Logistic Regression model achieved a high accuracy of 98.10% on the test dataset, slightly outperforming the Naive Bayes classifier.
- For both "Not spam" and "Spam" classes, the precision, recall, and F1-score values are high, indicating the model's ability to correctly identify both classes with high accuracy and efficiency.
- The classification report suggests that the Logistic Regression model performs exceptionally well in distinguishing between "Not spam" and "Spam" messages, with high precision and recall values for both classes.

5.3 Simple RNN

```
Epoch 1/5
146/146 [=====] - 45s 270ms/step - loss: 0.4146 - accuracy: 0.8132 - val_loss: 0.1371 - val_accuracy: 0.9599
Epoch 2/5
146/146 [=====] - 29s 195ms/step - loss: 0.0879 - accuracy: 0.9774 - val_loss: 0.0764 - val_accuracy: 0.9721
Epoch 3/5
146/146 [=====] - 23s 159ms/step - loss: 0.0505 - accuracy: 0.9882 - val_loss: 0.0473 - val_accuracy: 0.9808
Epoch 4/5
146/146 [=====] - 22s 151ms/step - loss: 0.0406 - accuracy: 0.9899 - val_loss: 0.0522 - val_accuracy: 0.9808
Epoch 5/5
146/146 [=====] - 21s 144ms/step - loss: 0.0226 - accuracy: 0.9940 - val_loss: 0.0608 - val_accuracy: 0.9808
19/19 [=====] - 1s 12ms/step

Simple RNN Accuracy: 0.9793103448275862
```

- The model shows significant improvement in accuracy and reduction in loss across epochs, indicating the model's learning and improvement over successive iterations.
- The training accuracy consistently increases while the loss decreases with each epoch, demonstrating that the model is learning to predict better on the training data.
- Validation accuracy and loss also show improvements, suggesting that the model is generalizing well to unseen data (validation set) and not overfitting.
- The slight difference between validation and training accuracy/loss in some epochs indicates the model's ability to generalize to new data (validation) while still performing well on the training set.
- The final accuracy on the test set (97.93%) indicates the model's ability to make accurate predictions on previously unseen data.

5.4 LSTM

```
Epoch 1/5
146/146 [=====] - 20s 117ms/step - loss: 0.2786 - accuracy: 0.8763 - val_loss: 1.3337 - val_accuracy: 0.6725
Epoch 2/5
146/146 [=====] - 7s 52ms/step - loss: 0.1195 - accuracy: 0.9582 - val_loss: 0.0612 - val_accuracy: 0.9739
Epoch 3/5
146/146 [=====] - 6s 39ms/step - loss: 0.0750 - accuracy: 0.9739 - val_loss: 0.1015 - val_accuracy: 0.9669
Epoch 4/5
146/146 [=====] - 5s 32ms/step - loss: 0.0666 - accuracy: 0.9741 - val_loss: 0.0476 - val_accuracy: 0.9774
Epoch 5/5
146/146 [=====] - 3s 22ms/step - loss: 0.0418 - accuracy: 0.9838 - val_loss: 0.0630 - val_accuracy: 0.9739
19/19 [=====] - 0s 3ms/step

LSTM Accuracy: 0.9672413793103448
```

- The initial training epoch starts with a lower validation accuracy, which gradually increases over epochs.
- There's an increase in training accuracy over epochs, indicating that the model is learning and improving on the training data.
- Validation accuracy generally shows good performance, indicating the model's ability to generalize to unseen data.
- The test set accuracy of 96.72% indicates the model's performance on previously unseen data (test set), demonstrating its ability to make accurate predictions.

6 Test

Example 1: Congratulations! You've won a free vacation. Click here to claim your prize.

Naive Bayes Prediction: spam

Logistic Regression Prediction: spam

1/1 [=====] - 0s 22ms/step

LSTM Prediction: spam

1/1 [=====] - 0s 30ms/step

Simple RNN Prediction: spam

- All four models (Naive Bayes, Logistic Regression, LSTM, and Simple RNN) classify this message as "spam."
- The message likely contains phrases often associated with spam content, such as "Congratulations," "free vacation," and "claim your prize," leading the models to categorize it as "spam."

```
Example 2: Hi,I am Ahmed.  
Naive Bayes Prediction: not spam  
Logistic Regression Prediction: spam  
1/1 [=====] - 0s 23ms/step  
LSTM Prediction: not spam  
1/1 [=====] - 0s 30ms/step  
Simple RNN Prediction: spam
```

1. **Naive Bayes:** This model predicted the message as ‘not spam’. Naive Bayes classifiers are based on applying Bayes’ theorem with strong independence assumptions between the features. In the context of spam detection, it considers each word in the email independently and estimates the probability of spam given the presence of that word. It seems to have correctly identified that the message “Hi, I am Ahmed.” is not spam.
2. **Logistic Regression:** This model predicted the message as ‘spam’. Logistic Regression measures the relationship between the categorical dependent variable and one or more independent variables by estimating probabilities using a logistic function. The model predicted this simple greeting as spam, it might be due to some bias in the training data where similar phrases were labeled as spam.
3. **LSTM (Long Short-Term Memory):** This model predicted the message as ‘not spam’. LSTM is a type of recurrent neural network that can learn order dependence in sequence prediction problems. This is important in spam detection as the meaning of a word can often depend on its context and order in the sentence. It seems to have correctly learned that the message “Hi, I am Ahmed.” is a typical non-spam greeting.

4. **Simple RNN (Recurrent Neural Network):** This model predicted the message as 'spam'. Simple RNNs also have the ability to use their internal state (memory) to process sequences of inputs, but they can struggle with long-term dependencies due to the vanishing gradient problem. This might have led to the incorrect classification.